21 不相交集合

Part V Advanced Data Structures

本章定位

不相交集合把只合不分的partition轉成代表查詢；本章以同一組合併對照weighted list與rank forest，再由完整level/iter位能計費推導反Ackermann總界，區分單次成本、序列攤銷與實作前提。

前置知識

Representative、森林、rank 與路徑壓縮。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較 linked-list 與 forest 表示。
- 用 union by rank 與 path compression。
- 理解 inverse Ackermann 攤銷界。
- 21.1 Disjoint-set operations
- 21.2 Linked-list representation of disjoint sets
- 21.3 Disjoint-set forests
- 21.4 Analysis of union by rank with path compression

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.1 Disjoint-set operations

維護的是 partition，代表元素只是 handle

Universe中的每個已建立元素恰屬一個非空集合，集合兩兩不交。MAKE-SET(x)要求x尚未建立，產生{x}；FIND-SET(x)要求x存在，回傳其集合的一個代表；UNION(x,y)將兩個集合合成一個。代表不自動等於最小鍵，內部parent亦不是原圖的邊。

本章令n為MAKE-SET次數，m為包含MAKE-SET、FIND、UNION的全部API呼叫數，m≥n；為固定分析參數，先完成n個單點再操作。成功合併最多n−1次，但重複UNION API可任意多。實作先取兩代表，若相同直接返回，不得把同一根重新LINK或增加rank。教材採此可重複呼叫的契約，LINK則只接受**不同的兩個根**。

FIND可能壓縮parent但不改partition；UNION只合不分。刪除元素、撤銷合併、動態刪圖邊或輸出全部成員，均不包含於三操作的既有保證，須另設結構。不同集合只要求代表不同，同一集合的所有FIND必回同一代表。

連通分量：為何合併後既不漏也不誤合

對無向圖，先MAKE-SET每個頂點；每來一條(u,v)就UNION(u,v)。不變量：處理的邊所形成子圖，其連通分量恰等於DSU partition。初始只有單點；新邊兩端同分量時兩者都不變，不同分量時恰用該邊接成一個分量，所以歸納成立。結束後FIND(u)=FIND(v)⇔存在原圖無向u-v路。

六點a..f依序加入ab、cd、bc、ab、ef、de：分量依序{ab},{c},{d},{e},{f}；{ab},{cd},{e},{f}；{abcd},{e},{f}；重複ab不變；{abcd},{ef}；{abcdef}。成功合併5次，API UNION有6次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連通分量：為何合併後既不漏也不誤合 （續）

邊串流：partition 的每次改變

每列為處理新邊後的partition；第4列重複邊不改集合數。

			集合數
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

成功5次合併：6→5→4→3→3→2→1；全部6次 UNION 仍各需做代表檢查。

靜態圖DFS/BFS已可Θ(n+|E|)找分量；DSU的用途還包括邊逐一加入時即時查連通，或Kruskal辨別新邊是否成cycle。有向圖若忽略方向合併只得到弱連通，不是SCC。

21.2 Linked-list representation of disjoint sets

Set record 與每個成員的反向指標

每集合record存head、tail、size；每成員存next及set指標。代表是record.head的成員。空collection可O(1)建立；MAKE-SET產生單一節點，next=NIL、head=tail=x、size=1、x.set=新record。FIND只回x.set.head，O(1)。

UNION取得S=x.set、T=y.set；若相同返回head。若S.size<T.size交換S,T，同大保留第一個S。將S.tail.next=T.head、S.tail=T.tail、S.size+=T.size，再逐一走T原串列，把每個成員的set改成S，最後廢棄舊record T。串列next順序未在走訪中改變，因此不會遺失後繼；不能只接tail卻不更新set。

一次UNION成本Θ(1+min(|S|,|T|))，不是O(1)。例如長串列a→b→c、短串列d→e，合併後a→b→c→d→e，僅d,e的set改指長record；tail由c改e、size由3改5，d/e的FIND才會回a。

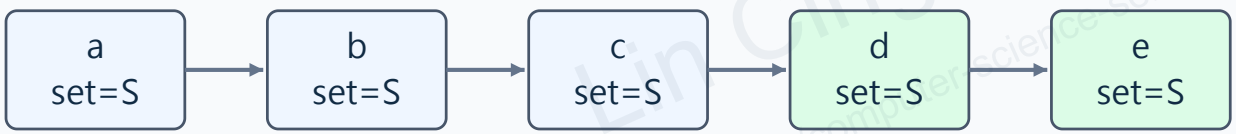
21.2 Linked-list representation of disjoint sets （續）

Weighted list：接 next 之外還要改 set

合併前 S: head=a, tail=c, size=3；T: head=d, tail=e, size=2。



短串列接到長尾；只改 d,e 的 set → S，並令 c.next=d。



合併後 S: head=a, tail=e, size=5；綠格 = 本次改set的成員；箭頭 = next。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不加權的平方反例與加權的 doubling 證明

若總把第二個集合接到第一個尾端，先n個單點，再做UNION(x_2,x_1)、UNION(x_3,x_2)、…，每次新的單點吸收之前整個長串列。改寫成員數依序1,2,…,n−1，合計n(n−1)/2；m=2n−1卻需Θ(n^2)，所以「有tail可O(1)串接」不等於整個UNION O(1)。

Weighted union每次只移短集合。固定成員x每次set改寫，所在集合大小至少倍增；改k次後至少2^k個成員，最多n，故k≤floor($\log_2 n$)。加總n個成員最多n floor($\log_2 n$)次改寫；每次API常數開銷另計m，總O(m+n log n)，小n用log(n+1)。單次兩個各n/2集合合併仍Θ(n)，不能把此總界解讀成每次最壞O(log n)。

上界亦有對應成長序列：$n=2^k$時，每輪成對合併同大小集合，k輪每輪恰改寫n/2個set，共(n/2)$\log_2 n$。MAKE/FIND的O(1)與整段總成本可以同時成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八元素的每次改寫帳

同大小保留第一集合。下列次序亦供森林版本使用；字母只是handle，不作大小比較。

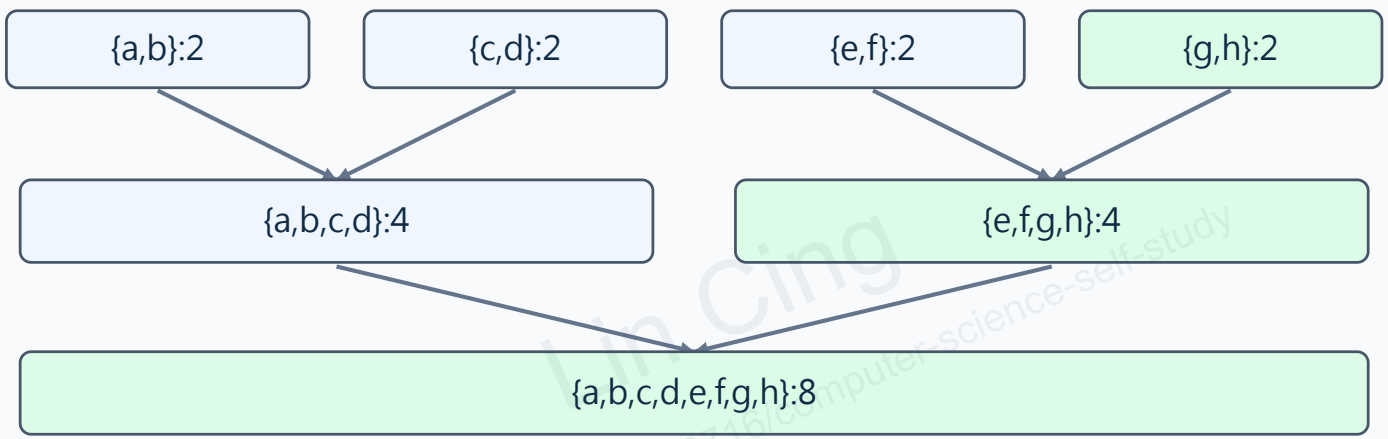
UNION	原大小	改set的成員	新代表／大小	累計改寫
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

h的set依h→g→e→a，改寫時新集合大小2→4→8，恰三次；a完全不必改。所有成員的次數加總12，並非每個成員都三次。若總以單點接到大集合的加權版本，前述平方反例會變成只有n−1次成員改寫。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

成對同大合併：三輪的逐人改寫帳

每輪改寫4個set；綠框是 h 所在集合，大小2→4→8。



三輪總改寫 4+4+4=12；h改3次，a改0次；集合不分裂。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root 自指與 LINK 的前提

每元素存parent、rank，根parent指向自己。MAKE-SET令parent[x]=x、rank[x]=0。UNION不是把任意x直接接到y，而是先找兩根；否則可能切斷原集合或製造cycle。

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    若 r==s: 回傳 r
    回傳 LINK(r,s)
LINK(r,s):                      // r,s 是不同的根
    若 rank[r]<rank[s]: 交換 r,s
    parent[s]=r
    若 rank[r]==rank[s]: rank[r]+=1
    回傳 r
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

接根後整個舊集合仍沿parent到新根；只有一個root失去自指，沒有反向可回的路，所以仍為森林。不同rank不增加新根rank；相同rank只使保留下來的新根加一。這是union by rank，與linked list按size比較是兩種不同表示法，不能把rank當精確成員數。

兩趟 FIND：覆寫前保存舊父

```
FIND(x):
    r=x
    當 parent[r]!=r: r=parent[r]
    當 parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    回傳 r
```

第一趟找根而不改任何指標；第二趟走原路，把沿途parent全部改r。若先覆寫parent[x]再用它繼續走，就會直接跳r而漏壓縮中間節點。遞迴寫法parent[x]=FIND(parent[x])等價於上行找根、返回時改連結；遞迴stack最多路徑高度，兩趟迭代輔助O(1)。

壓縮只把同樹節點改接其祖先根，根與partition都不變；不修改rank。一次走s個節點成本Θ(s)。森林本身O(n)空間，若還要列全部成員，可另維護成員串列；單靠parent不會自動提供O(集合大小)列舉。

同一八元素的 rank 與壓縮前後

沿前述七次UNION，同rank保留第一根，根rank依a1,c1,e1,g1,a2,e2,a3。最終parent為b→a、c→a、d→c、e→a、f→e、g→e、h→g；a自指。

點	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
FIND(h)前parent	a	a	a	c	a	e	e	g
FIND(h)後parent	a	a	a	c	a	e	a	a

第一趟h→g→e→a三跳，第二趟h.parent=a、g.parent=a、e.parent=a（最後一個原本已如此）。d→c與f→e不在此find path，不改；rank全部保持。再FIND(h)只一跳，但FIND(d)仍兩跳，不能把一次find說成全樹扁平化。

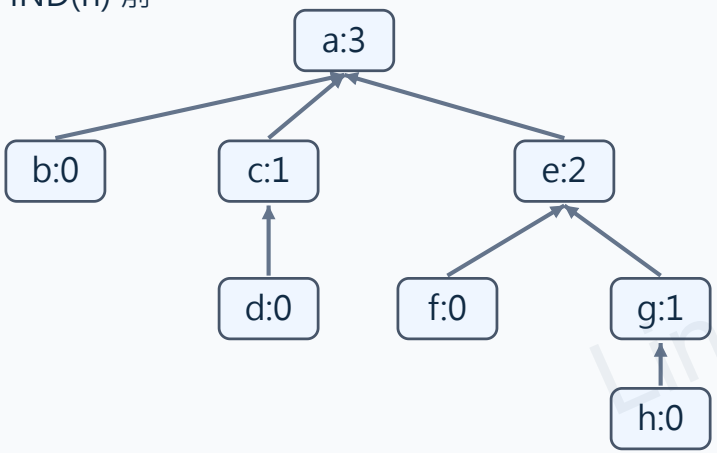
Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一八元素的 rank 與壓縮前後 （續）

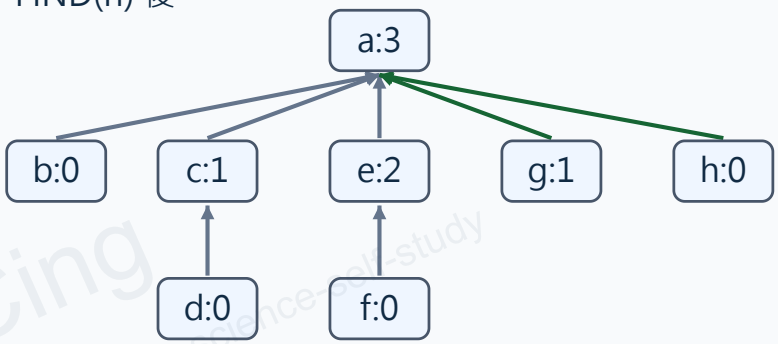
同一八點樹：壓縮前後的完整 parent

節點 = 鍵:rank；箭頭 = parent（根自指省略）；綠線 = 本次改寫。

FIND(h) 前



FIND(h) 後



h→g→e→a；只改 h,g 的實際連線，e 本就指 a；d,f 不在路徑。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Rank 的高度上界與歷史成員下界

對操作數歸納，root的目前高度≤其rank：單點為0；不同rank時小樹高度+1≤大rank，相同rank時最多原rank+1；壓縮只會縮短路徑，不增加高度。根rank=r的集合至少2^r個成員：只在兩個rank r−1根合併時升級，兩集合各至少2^{r-1}；其他合併只增加成員。非根的rank固定，沿用它最後為根時所在集合的歷史下界，因此任一rank≤floor($\log_2 n$)。

Rank不是目前高度的下界。例中a.rank=3，但壓縮全部元素後根只有高度1；rank=2的根亦能在四點星狀結構中高度1。它也不是目前subtree大小，因壓縮會移走後代，卻不減rank。

不用壓縮、只by-rank，單次FIND O(log n)，全部m次O(m log n)；成對合併出高度$\log_2 n$，再對深葉重複FIND便達對應量級。沒有by-rank時可先LINK成線性長鏈，使第一次FIND Θ(n)；不能把這種反例套到同時使用兩heuristics的定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.4 Analysis of union by rank with path compression

Ackermann 的迭代不是普通次方

本節固定最終n與α(n)，在同一操作序列中不隨當前集合數改變α。記$f^{(0)}(j)=j$、$f^{(i+1)}(j)=f(f^{(i)}(j))$。對k≥0、j≥1，$A_0(j)=j+1$，$A_k(j)=A_{k-1}^{(j+1)}(j)$。上標是**函數合成次數**。

由$A_0^{(i)}(j)=j+i$得$A_1(j)=2j+1$；再對i歸納$A_1^{(i)}(j)=2^i(j+1)-1$，令i=j+1得$A_2(j)=2^{j+1}(j+1)-1$。所以$A_3(1)=A_2(A_2(1))=A_2(7)=2047$。

單調性亦由歸納建立：A_0嚴格遞增且$A_0(j)\geq j+1$；若A_{k-1}具有兩性質，從較大j開始並多迭代一次，結果必更大，故A_k對j嚴格遞增。又j+1≥2，至少兩次迭代給$A_k(j)>A_{k-1}(j)$；並有$A_k(j)\geq j+1$。以下level界與好節點不等式使用的正是這些單調性。

Lin Ching https://github.com/xinhi6716/computer-science-self-study

所用 inverse-Ackermann 版本

本章採用 Cormen 第三版 21.4 的單參數版本。對 `k>=0`、`j>=1` 定義：

- `A_0(j)=j+1`
- `A_k(j)=A_{k-1}^{(j+1)}(j)`，右上角表示把 `A_{k-1}` 迭代 `j+1` 次。
- `alpha(n)=min{k>=0 | A_k(1)>=n}`。

這與常看到的雙參數版本不同。`A_0(1)=2`、`A_1(1)=3`、`A_2(1)=7`、`A_3(1)=2047`、`A_4(1)>10^80`，所以 `alpha(2)=0`、`alpha(8)=3`、`alpha(2048)=4`。

定理的等價寫法會保留小 n 的基例：

`m` 次操作、`n` 次 MAKE-SET 的總成本為 `O(m(1+alpha(n)))`。

當 `n>=3` 時 `alpha(n)>=1`，才可化簡成教科書常見的 `O(m alpha(n))`。若直接對 n=0,1,2 寫 `O(m alpha(n))`，會錯誤地把上界寫成 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

所用 inverse-Ackermann 版本 （續）

Ackermann 門檻與固定倍率的位能總帳

固定最終 n；$\alpha=\min\{k\ge 0: A_k(1)\ge n\}$；$\Psi=C\Phi$，$C>c$ 為固定正常數。

單參數門檻 $A_k(1)$	同一 Ψ 支付三種操作
$A_0(1)=2$	MAKE：新rank0，$\Delta\Psi=0$，成本 O(1)。
$A_1(1)=3$	LINK：$\Delta\Psi\le C\alpha$，實際成本 O(1)。
$A_2(1)=7$	FIND：s個點，$g\ge\max(0,s-(\alpha+2))$。
$A_3(1)=2047$	$\Delta\Psi\le -Cg$；$cs-Cg\le \max(c,C)(\alpha+2)$。
$A_4(1)>10^{80}$	Ψ初始0、終值≥0 ⇒ Σ實際 ≤ Σ攤銷。

$\alpha(2)=0$、$\alpha(8)=3$、$\alpha(2048)=4$；含小n基例的總界為 $O(m(1+\alpha(n)))$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

rank 與路徑的不變量

MAKE-SET 建立 rank 0 的單點。LINK 把 rank 小的根接到 rank 大的根；rank 相等時選定一個新根並把 rank 加一。FIND-SET 的 path compression 只改 parent，不改 rank。

rank 單調。 本節的圖論記法約定根 x 的 `x.parent=x`；實作若以 NIL 表示根，只需把 parent 判空改寫成同一語意。對操作次數作歸納，可證：

- 根 x 的 parent 指向自己，且其 rank 只在兩個同 rank 根合併時增加一；`MAKE-SET` 建立 rank 0 的根；
- 若 x 不是根，則 `x.rank < x.parent.rank`；
- x 一旦不再是根，其 rank 固定；父節點的 rank 只會增加。

因此任一條 find path 從 x 往根走時，rank 嚴格增加。

rank 的範圍。 對根 x 歸納可證以 x 為根的集合至少 `2^{x.rank}` 個元素。當兩個 rank r 根合併時，總大小至少 `2^r+2^r=2^{r+1}`；不同 rank 合併時較大根的大小不減。非根則沿用它失去根身分時的歷史rank下界。故 `rank(x)<=floor(log_2 n)<=n-1`。

對非根 x，`level(x)` 只看目前父 rank，因此父 rank 單調使 level 不會下降。`iter(x)` 則可能因 level 上升而重設並下降；不能寫成 level 與 iter 都不下降。

level、iter 與位能

令 alpha=alpha(n)。對非根且 r=x.rank>=1 的節點，定義：

- level(x)=max{k | x.parent.rank>=A_k(r)}
- iter(x)=max{i | x.parent.rank>=A_level(x)^(i)(r)}

由 A_0(r)=r+1 與父 rank 不小於 r+1，得 level(x)>=0。對 n>=3，A_alpha(1)>=n>x.parent.rank，而 A_alpha(r)>=A_alpha(1)，故 level(x)<alpha。

iter(x)>=1 來自 x.parent.rank>=A_level(x)(r)。若 iter(x)>=r+1，依迭代定義

x.parent.rank>=A_level(x)^(r+1)(r)=A_{level(x)+1}(r)，

與 level(x) 的最大性矛盾；所以 1<=iter(x)<=r。

定義各點位能並令 Phi=sum phi(x)：

```
phi(x) = alpha * r                          若 x 是根或 r=0
phi(x) = (alpha-level(x))*r - iter(x)       若 x 非根且 r>=1
```

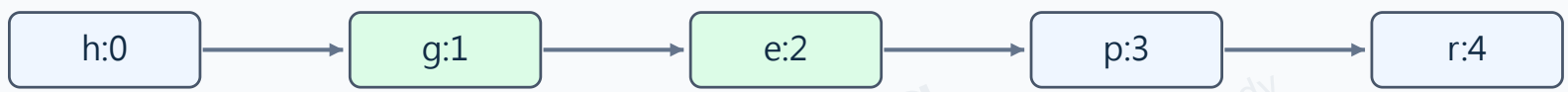
Lin Cing https://github.com/xinhi6716/computer-science-self-study

level、iter 與位能 （續）

第一分支是 `alpha` 乘 `r`，不是 `alpha-r`；因此 MAKE-SET 的 rank 0 點位能為 `alpha*0=0`。第二分支在 `0<=level<alpha`、`1<=iter<=r` 下滿足 `0<=phi(x)<alpha*r`。

FIND 的數值：level 與 iter 各自的前進

n=16, α=3；FIND(h) 後 h,g,e,p 都指根 r，rank 全部不變。



	h:0	g:1	e:2	p:3	r:4
前 (level,iter)	—	(0,1)	(0,1)	(0,1)	root
後 (level,iter)	—	(1,1)	(0,2)	(0,1)	root
	φ 0→0	φ 2→1	φ 5→4	φ 8→8	φ 12→12

綠節點 g,e 是好節點：本path位能27→25；rank0、root、最後同level點為例外。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

level 上升時 iter 可下降

固定 rank r。若 level 不變而 iter 改變，父 rank 增加只會使同 level 可迭代次數增加，故此情形 iter 不下降。

若 level 由 k 升到 k'>k，iter 可以由 i 降到 i'，甚至回到 1。位能差仍是

```
phi_after-phi_before
= -(k'-k)r + (i-i').
```

因為 k'-k>=1，且由 `1<=i'`、`i<=r` 得 `i-i'<=r-1`，所以差值至多 `-r+(r-1)=-1`。也就是說 (`level` 越大越有利，同 `level` 下 `iter` 越大越有利) 的 lexicographic 次序嚴格增進，`phi` 至少下降 1。

因此前節的位能同時處理「iter 在同 level 上升」與「level 上升造成 iter 重設」；不需要也不可能主張 iter 永不下降。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

FIND-SET 的好節點不等式

固定一次 path compression。設 x 在 find path 上，`r=rank(x)>=1`，且路徑後面有非根節點 y 滿足 `level(x)=level(y)=k`。當此 y 存在時稱 x 為好節點。

壓縮前，依三條定義與 rank 的單調性：

```
rank(y)>=rank(parent(x))
rank(parent(y))>=A_k(rank(y))
>=A_k(rank(parent(x)))
>=A_k^(iter(x)+1)(rank(x)).
```

路徑壓縮後，x 與 y 有同一個根，所以

`new parent(x).rank = new parent(y).rank >= old parent(y).rank >= A_k^(iter(x)+1)(rank(x))`。

若這個新界尚未觸發 level 上升，則 iter(x) 至少增加 1。若 iter 增加超過 r，依據 `iter<=rank` 的反證即 level 必須上升。故每一好節點在壓縮後都使 `(level,iter)` lexicographic 前進，前節已證明 phi 至少下降 1。

Lin Cing
github.com/xinhi6716/computer-science-self-study

FIND-SET 的常數級 charging

find path 有 s 個節點。例外至多 `alpha+2` 個：rank 0 的起點（若存在）、根，以及每個 level `0,...,alpha-1` 在路徑上的最後一個節點。所以好節點至少有

`g=max(0,s-(alpha+2))`。

每個好節點令 Phi 至少下降 1。其他 path 節點不使位能增加。若掃描常數使實際成本至多 `c s`，現在明確定義放大後的總位能 `Psi=C Phi`，其中 `C>c` 是本次分析固定的常數。乘正常數不改變前述 ranking 不等式；好節點使 Psi 至少下降 C，所以 `Delta Psi<=-Cg`。因此

```
amortized FIND = actual+Delta Psi
<= c*s-C*g
<= max(c,C)(alpha+2).
```

右側是 `O(alpha+2)=O(1+alpha)`，不是粗糙的 `O(s)-Omega(s-alpha)=O(alpha)` 直接相消。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

FIND-SET 的常數級 charging （續）

MAKE-SET 的實際成本 O(1)，新點 rank 0 的 Phi 為 0，所以 Psi 為 0。LINK 的實際成本 O(1)；附上的舊根失去root身分，位能不增；新根原有child因父rank不降，位能不增；只有新根可能在同 rank 合併時升一級，且 Phi 的增量至多 alpha，所以 Psi 的增量至多 `c alpha`。LINK 的攤銷成本是 `O(1)+c alpha=O(1+alpha)`。因此 MAKE-SET 為 O(1)，LINK 為 `O(1+alpha)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

總成本與適用範圍

把 UNION 轉成兩次對根的 FIND-SET 再加一次 LINK，操作數最多乘 3，不改變 O 級。把每次操作的實際成本記為 c_i、攤銷成本記為

$a_i=c_i+\Delta \Psi_i$。

對整段序列作 telescoping：

$$\sum_i a_i = \sum_i c_i + \Psi_{final}-\Psi_{initial}.$$

所以

$\sum_i c_i = \sum_i a_i-\Psi_{final}+\Psi_{initial}$。

初始資料結構為空，故 $\Psi_{initial}=0$；所有 $\phi(x)$ 非負，故 $\Psi_{final}\geq 0$。於是 $\Psi_{initial}-\Psi_{final}\leq 0$，整段實際成本滿足

$\sum_i c_i\leq \sum_i a_i$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

總成本與適用範圍 (續)

這一步把每次操作的攤銷上界轉成整段操作序列的實際上界。對轉換後序列套用前述 amortized FIND/LINK 界：

`O(m(1+alpha(n)))`。

再轉回原序列仍是同一個界。當 `n>=3` 時，`alpha(n)>=1`，才寫成 `O(m alpha(n))`；`n=0,1,2` 另由 `alpha=0` 的基例處理，不能把總工作量誤寫成 0。

這是整段操作序列的最壞情形上界，不是每次 FIND-SET 都最壞 O(alpha(n))。在 union by rank 下單次 FIND-SET 的路徑高度為 O(log n)，不需要也不能宣稱在該設定中仍存在 Omega(n) 長鏈；若移除 union-by-rank，任意連結才可能形成線性長鏈，但那就不是本章定理的設定。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 21。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §21.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study